

LE GRAND GOURMET

Site web communautaire de recettes de cuisine — récapitulatif du dossier de projet et du dossier professionnel

CANDIDAT

Alexandre BAPECK

ÉTABLISSEMENT

La Plateforme — Cannes

TITRE VISÉ

Développeur Web et Web Mobile

STACK

PHP 8 · PDO · MySQL · HTML5 · CSS3 · JS

NIVEAU

RNCP Niveau 5 — Bac +2

ENVIRONNEMENT

XAMPP · VS Code · Git / GitHub

1 Le projet en une page

LE GRAND GOURMET est une application web complète de partage de recettes : un visiteur parcourt et recherche le catalogue, un membre inscrit publie, note, commente et sauvegarde en favori, un administrateur gère le contenu et modère la communauté depuis un back-office. Projet fil rouge personnel, développé de bout en bout en **PHP natif sans framework** — un choix assumé, destiné à prouver la maîtrise des mécanismes fondamentaux (cycle requête/réponse, sessions, sécurité) plutôt qu'à les masquer derrière une couche d'abstraction.

8

TABLES

19

USER STORIES

3

MISSIONS

~50

FONCTIONS PHP

3

RÔLES

2 Les acteurs et leurs droits

RÔLE	DROITS ET ACCÈS
Visiteur	Consultation du catalogue, recherche, filtres, lecture des recettes et commentaires. Aucun compte requis.
Utilisateur	Droits du visiteur + publication de recettes, notation 1-5, commentaires, favoris, gestion de son profil.
Administrateur	Tous droits + back-office : CRUD recettes / catégories / utilisateurs et modération des commentaires.

Héritage des droits : l'utilisateur inscrit hérite des cas d'utilisation du visiteur, l'administrateur de ceux de l'utilisateur. Les gardes `requireLogin()` et `requireAdmin()` verrouillent l'accès en première ligne de chaque page protégée.

3 Architecture et stack technique

COUCHE	TECHNOLOGIE
Serveur web	Apache 2.4 (XAMPP)
Langage serveur	PHP 8.x
Accès données	PDO (requêtes préparées)
Base de données	MySQL / MariaDB 10 — InnoDB
Structure	HTML5 sémantique
Style	CSS3 — variables, Grid, media queries
Interactivité	JavaScript ES6 vanilla (fetch)
E-mail	PHPMailer 6 (SMTP)

Séparation en trois couches

- **Présentation** — pages, `includes/`, `assets/`
- **Métier** — `admin/`, `ajax/`, traitements en tête de page
- **Données** — `database.php` (PDO) et `fonctions.php`

Gestion de projet

- Développeur unique, plusieurs casquettes (conception, front, back, tests)
- Approche itérative et incrémentale : socle BDD → authentification → interactions → back-office
- Git / GitHub, GitHub Projects (Kanban), commits conventionnels (`feat`, `fix`, `docs`)
- Composer pour les dépendances (PHPMailer)

4 Charte graphique et responsive

#C8603A Terracotta

#3E2C23 Brun foncé

#FBF6EF Crème

#7C8B6F Sauge

#E8B84B Miel / or

Typographies : **Playfair Display** (serif, titres) et **Inter** (sans-serif, corps de texte) — contraste serif/sans-serif classique du design éditorial. Maquettes Figma en **mobile-first**, enrichies pour tablette et desktop. Grille de recettes adaptative : 1 colonne par défaut, 2 colonnes à partir de 600 px, 3 colonnes à partir de 900 px. Navigation repliée derrière un menu burger animé sur petit écran.

5 Base de données — 8 tables

Conception selon la démarche **Merise** en trois niveaux : MCD (conceptuel), MLD (logique, où les associations many-to-many deviennent des tables) puis MPD (physique, types SQL réels). Moteur InnoDB, encodage utf8mb4.

TABLE	RÔLE
utilisateurs	pseudo, email, mot_de_passe (haché), role
categories	Entrées, Plats, Desserts...
recettes	titre, slug, instructions, difficulté
ingredients	Liste sans doublon
recette_ingredient	Pivot + quantité (N-N)
commentaires	Avis des membres
favoris	Pivot utilisateur ↔ recette
notes	Valeur 1-5, <code>UNIQUE(user, recette)</code>

Intégrité référentielle

- Une catégorie classe 0..N recettes ; une recette appartient à 1 seule catégorie.
- Une recette a 1 auteur ; un utilisateur écrit 0..N recettes.
- Recettes ↔ ingrédients : many-to-many résolu par la pivot `recette_ingredient`, porteuse de l'attribut `quantite`.
- `ON DELETE CASCADE` pour les données dépendantes (commentaires, notes, favoris).
- `ON DELETE RESTRICT` pour empêcher la suppression d'une catégorie encore utilisée.
- Contraintes `UNIQUE` (une note par membre et par recette) et `CHECK (valeur BETWEEN 1 AND 5)`.

ACID : InnoDB garantit l'**A**tomicité (recette + ingrédients forment un tout), la **C**ohérence (pas de commentaire orphelin), l'**I**solation (deux notes simultanées ne se corrompent pas) et la **D**urabilité (les données survivent à un arrêt serveur).

6 Les trois missions techniques

Mission 1 — CRUD des recettes en back-office

Gestion complète des recettes par l'administrateur (créer, lister, modifier, supprimer) avec upload d'image et édition dynamique des ingrédients. Séparation MVC sans framework : la **vue** et le **contrôleur** cohabitent dans `admin/recettes.php` (routing par `?action=`), le **modèle** vit dans `fonctions.php`. Validation doublée : les attributs HTML (`required`, `min`) pour le confort utilisateur, la validation PHP côté serveur pour la sécurité réelle. Difficulté principale — la sauvegarde des ingrédients dynamiques — résolue par une fonction qui supprime puis réinsère les liens, en créant à la volée les ingrédients inconnus.

Mission 2 — Favoris en AJAX (mini-API JSON)

Ajout ou retrait d'un favori sans rechargement : le navigateur envoie une requête `fetch` POST à `ajax/favori.php`, qui renvoie un statut JSON et met à jour le cœur dans le DOM. Les favoris sont persistés **en base** (et non en `localStorage`), afin de suivre l'utilisateur sur tous ses appareils.

ÉLÉMENT	VALEUR
Endpoint	<code>POST /ajax/favori.php</code>
Authentification	Session PHP (cookie). Si absente : réponse de redirection.
Payload	<code>recette_id=<int></code> (x-www-form-urlencoded)
Réponses 200	<code>{ "status": "added" } · { "status": "removed" }</code>
Codes d'erreur	405 (méthode non autorisée) · 400 (<code>recette_id</code> manquant)

Mission 3 — E-mail transactionnel via SMTP

Un e-mail de bienvenue est envoyé automatiquement à l'inscription, via **PHPMailer** installé avec Composer et un serveur SMTP authentifié (**Mailtrap** en développement, qui capture les messages sans les envoyer réellement). Le protocole SMTP est préféré à la fonction native `mail()`, peu fiable et souvent classée en spam. Les identifiants ne sont jamais écrits en dur : ils sont lus depuis un fichier `.env` non versionné, documenté par un `.env.example`. En cas d'échec d'envoi, l'erreur est journalisée via `error_log()` sans bloquer l'inscription.

7 Sécurité — couverture du Top 10 OWASP

MENACE	PARADE IMPLÉMENTÉE	OÙ ?
Injection SQL	Requêtes préparées PDO + <code>EMULATE_PREPARES=false</code> ; aucune concaténation.	Toutes les fonctions BDD
XSS	Échappement systématique <code>e() = htmlspecialchars(ENT_QUOTES)</code> .	Tous les affichages
CSRF	Jeton aléatoire (<code>random_bytes</code>) en session, champ caché, vérif. <code>hash_equals()</code> .	Tous les formulaires POST
Mots de passe	Hachage bcrypt (<code>password_hash</code>) ; jamais en clair.	Inscription / connexion / profil
Fixation de session	<code>session_regenerate_id(true)</code> à l'authentification.	Connexion / inscription
Accès non autorisé	<code>requireLogin()</code> / <code>requireAdmin()</code> en tête de page.	Pages privées + admin
Upload malveillant	Extension + type MIME réel (<code>finfo</code>) + taille + nom régénéré.	Upload d'images
Secrets exposés	Identifiants en <code>.env</code> exclu du versionnement.	Config BDD et SMTP

Hachage ≠ chiffrement. Le chiffrement est réversible (on déchiffre avec une clé) ; le hachage est à sens unique. Un mot de passe ne doit jamais être déchiffrable : bcrypt intègre un sel aléatoire et un coût configurable, et à la connexion on compare les hachages sans jamais retrouver le mot de passe d'origine. L'emploi de `hash_equals()` (comparaison à temps constant) plutôt que `==` protège des attaques temporelles.

8 Tests et déploiement

SCÉNARIO	RÉSULTAT
Inscription e-mail déjà utilisé	Erreur, pas de doublon ✓
Connexion mauvais mot de passe	Refus, message générique ✓
Accès <code>/admin</code> sans rôle admin	Redirection + message ✓
POST sans jeton CSRF	Rejet sécurité ✓
Note hors plage (0 ou 6)	Ignorée (validation 1-5) ✓
<code>.exe</code> renommé <code>.jpg</code> en upload	Rejet (type MIME réel) ✓
E-mail de bienvenue	Reçu dans Mailtrap ✓

Procédure d'installation locale

- `git clone` dans `C:\xampp\htdocs\legrandgourmet`
- `composer install` (installe PHPMailer)
- Démarrer Apache + MySQL depuis XAMPP
- phpMyAdmin → Importer `database/legrandgourmetbdd.sql`
- `cp .env.example .env`, renseigner Mailtrap
- Ouvrir `http://localhost/legrandgourmet/`

Développement (XAMPP, Mailtrap) et production (serveur réel, SMTP réel) sont distingués **par la configuration, pas par le code**. Un jeu de données initial (seed) permet de tester l'application immédiatement après installation.

9 Compétences du référentiel — dossier professionnel

Le dossier professionnel documente la pratique sur les deux activités-types du titre, à travers des exemples issus du projet fil rouge et de projets d'équipe menés à La Plateforme (Cannes).

Activité-type 1 — Développer la partie front-end d'une application web sécurisée

EXEMPLE	CONTENU
LE GRAND GOURMET	Charte graphique Figma (terracotta / brun / crème), intégration HTML5 sémantique, CSS3 (variables, Flexbox, Grid), mobile-first. JavaScript : menu burger, favoris AJAX, notation étoiles. Responsive validé sur 3 breakpoints.
Festival Mars IA	Équipe de 5. Réflexion UX/UI collective, wireframes Figma, identité visuelle, intégration HTML/CSS responsive. Co-créditation La Plateforme × Mobile Film Festival.

Activité-type 2 — Développer la partie back-end d'une application web sécurisée

EXEMPLE	CONTENU
LE GRAND GOURMET	MCD / MLD / MPD, 8 tables. SQL : <code>CREATE TABLE</code> , <code>FOREIGN KEY</code> , <code>ON DELETE CASCADE</code> , contraintes <code>UNIQUE</code> et <code>CHECK</code> . Authentification bcrypt, rôles, CSRF. CRUD back-office, upload sécurisé, requêtes préparées.
Médiathèque PHP/MySQL	Équipe de 4, 20 jours ouvrés. Catalogue livres/films/jeux, recherche, filtres, pagination. Système d'emprunt (3 max, 14 j). Upload contrôlé (MIME + 2 Mo). CRUD en requêtes préparées.
Les Repas de Lili	Équipe de 8. React : composants fonctionnels, props, <code>useState</code> , rendu conditionnel.

Moyens et outils cités

Figma · HTML5 · CSS3 · JavaScript ES6 · API Fetch · Chrome DevTools · PHP 8 · PDO · MySQL 8 · phpMyAdmin · XAMPP · Visual Studio Code · React · Trello · Git / GitHub · Google Fonts. Travail en autonomie sur le projet fil rouge, en équipe sur les autres projets (points quotidiens, revues de code croisées, suivi Kanban), sous la supervision des formateurs de La Plateforme.

10 Bilan et perspectives

L'ensemble des fonctionnalités prévues au MVP a été livré et testé : catalogue avec recherche et filtres, authentification sécurisée, notation, commentaires, favoris en AJAX, publication de recettes et back-office complet. Plusieurs points ont été corrigés en cours de route — le hash administrateur du script SQL était un placeholder (régénéré avec un vrai bcrypt), le chemin d'upload du back-office a été fiabilisé, et la validation des filtres durcie par liste blanche.

La principale difficulté — garantir une sécurité réelle à chaque point d'entrée (formulaires, upload, sessions) — a été surmontée en adoptant systématiquement le réflexe « **ne jamais faire confiance aux données entrantes** » : validation côté serveur, requêtes préparées et échappement à l'affichage.

Pistes d'amélioration assumées

- Migration vers une architecture MVC objet (classes, autoloading, routeur)
- Conteneurisation Docker
- Tests unitaires automatisés (PHPUnit)
- API REST complète pour une future application mobile
- Réinitialisation de mot de passe par e-mail (jeton à durée limitée), réutilisant la brique SMTP
- Déploiement en ligne avec séparation dev / staging / prod

Mot de la fin — LE GRAND GOURMET constitue une application web complète et cohérente, qui démontre la maîtrise de toute la chaîne de développement, du modèle de données à l'expérience utilisateur, avec une attention constante à la sécurité.